

Local Variable Type Inference

(LVTI)

Definizione e contesto

Java è un linguaggio fortemente tipizzato (**strongly typed**). Questo significa che, di norma, quando dichiari una variabile sei obbligato a specificare esplicitamente il suo tipo. Per esempio:

```
String nome = "ABC";
```

```
int numero = 5;
```

```
double prezzo = 4.5;
```

A sinistra scrivi sempre il tipo (String, int, double).

A partire da Java 10 è stata introdotta una novità: in alcune situazioni puoi usare la parola chiave **var** al posto del tipo esplicito. In questi casi è il **compilatore** che "deduce" (inferisce) il tipo guardando il valore che assegna. Da qui il nome *Local Variable Type Inference*: inferenza del tipo per le variabili **locali**.

Esempio base:

```
var x = "ABC"; // il compilatore deduce che x è di tipo String
```

```
var y = 3; // il compilatore deduce che y è di tipo int
```

```
var z = 3.2; // il compilatore deduce che z è di tipo double
```

Il punto cruciale (e il tema di tutta la lezione) è: **var funziona solo con variabili locali e devono essere inizializzate nella stessa riga in cui vengono dichiarate**. Se dai al compilatore un valore, lui capisce il tipo; se non glielo dai, non può inferire nulla.

La lezione approfondisce questo concetto attraverso **7 regole**. Le vediamo una per una.

Dove si può usare var: solo in ambito locale

var può essere usato **solo all'interno di blocchi di codice**, cioè dove vivono le variabili locali:

- dentro i **metodi**
- dentro i **costruttori**
- dentro i **blocchi di inizializzazione**

Esempio commentato dalla lezione:

```
public class Esempio {
```

```
{
```

```
    var d = 4.5; // blocco di inizializzazione di istanza → d è un double ✓
```

```

}

public Esempio() {
    var cm = 9; // dentro un costruttore → cm è un int ✓
}

public void main() {
    var s = "ABC"; // dentro un metodo → s è una String ✓
}
}

```

Tutti e tre i casi sono **validi** perché ci troviamo in un contesto locale, dove il compilatore può fare l'inferenza del tipo.

Dove NON si può usare var

var **non** è consentito come:

- **parametro di costruttore**
- **parametro di metodo**
- **variabile di istanza** (dichiarata nella classe, fuori dai metodi)
- **variabile statica di classe**

Il motivo è sempre lo stesso: nessuno di questi è una variabile locale.

```

public class Esempio {

    var istanza = 5; // ✗ variabile di istanza → NON consentito

    static var classe = 5; // ✗ variabile statica → NON consentito

    public Esempio(var p) { } // ✗ parametro di costruttore → NON consentito

    public void metodo(var x) { } // ✗ parametro di metodo → NON consentito
}

```

Nota interessante sul parametro di metodo: anche se chiamassi `metodo(1)` passando un intero, il compilatore **non** deduce che `x` sia un `int`. Non è consentito e basta.

L'eccezione (avvertenza): le lambda

C'è un caso particolare, le **espressioni lambda**, in cui il tipo del parametro può essere dedotto. Esempio con una lista di stringhe:

```
List<String> lista = new ArrayList<>();  
lista.add("A");  
lista.add("A");  
lista.add("B");  
  
lista.removeIf((var s) -> s.equals("A")); // qui var è ammesso
```

Il metodo `removeIf` accetta un **predicato** (un `Predicate`): un predicato è un metodo `test` che riceve un parametro e **restituisce un booleano** (`true` o `false`). Rimuove ogni elemento per cui il test risulta `true` (qui rimuove ogni "A"). Dato che stai lavorando su una lista di `String`, il compilatore sa che l'elemento è una stringa, quindi in questo contesto lambda usare `var` è legale.

Ma questa è un'eccezione: **normalmente, come parametro, `var` non si può usare**, e ricorda che i costruttori sono a tutti gli effetti dei metodi normali.

Dichiarazione e inizializzazione devono stare nella stessa istruzione

Una variabile con `var` deve **sempre essere inizializzata nella stessa istruzione in cui viene dichiarata**.

```
var x = 5; // ✓ dichiarazione + inizializzazione insieme → x è un int
```

Cosa NON si può fare:

```
var x; // ✗ il punto e virgola chiude l'istruzione senza dare un valore  
x = 5; // troppo tardi: sono due istruzioni separate
```

Il punto e virgola conclude l'affermazione: se chiudi la dichiarazione senza aver assegnato un valore, il compilatore non ha nulla da cui dedurre il tipo. **Non puoi usare `var` su una variabile senza valore iniziale**.

Il valore può cambiare, il tipo no

Una volta che il compilatore ha inferito il tipo, quel tipo è **fisso**. Puoi cambiare il *valore* contenuto nella variabile, ma non il suo *tipo*.

```
var d = 4.5; // d è un double  
d = 98.9; // ✓ ho cambiato il valore, resta comunque un double
```

Quello che invece **non** puoi fare è assegnare un valore di tipo diverso:

```
var d = 4.5; // d è un double
```

```
d = "ciao"; // ❌ non puoi cambiare il tipo da double a String
```

In sintesi: è **possibile modificare il valore**, ma **non il tipo effettivo** della variabile.

Non si può inizializzare con `null`

Non puoi dichiarare una variabile `var` assegnandole `null`.

```
var g = null; // ❌ il compilatore non sa quale tipo dedurre
```

Il problema è concettuale: `null` è un valore che indica un **tipo di riferimento** qualsiasi, ma non dice *quale*. `g` dovrebbe essere una `String`? Un `Double`? Un `Integer`? Una `Persona`, un'Auto, un Cane? Il compilatore non ha modo di saperlo, quindi **non può inferire il tipo** e ti dà errore.

(Formalmente esisterebbe un modo con un cast esplicito, ma è un anti-pattern: se sai già il tipo, tanto vale scriverlo direttamente invece di usare `var`.)

`var` non è consentito nelle dichiarazioni multi-variabile

Non puoi dichiarare più variabili sulla stessa riga usando `var`.

```
var a = 3, b = 4; // ❌ dichiarazione multipla → NON consentito
```

Con i tipi espliciti questo sarebbe legale (`int a = 3, b = 4;`), ma con `var` no.

`var` è un nome riservato, non una parola riservata

Questa è una distinzione sottile ma importante. `var` **non è una keyword** (parola riservata) di Java: è un **reserved type name** (nome di tipo riservato). La conseguenza pratica è:

Puoi usare `var` come nome di identificatore (nome di variabile, di metodo, ecc.):

```
var var = 5; // ✔️ curiosamente legale: la prima 'var' è la keyword, la seconda è il nome
```

Ma **non puoi** usare `var` come nome di **classe, interfaccia o enum**:

```
class var { } // ❌ NON consentito
```

```
interface var { } // ❌ NON consentito
```

```
enum var { } // ❌ NON consentito
```

Se dai il nome `var` a una classe, un enum o un'interfaccia, hai un problema (errore di compilazione).